CS4423 - Networks

Prof. Götz Pfeiffer
School of Mathematics, Statistics and Applied Mathematics
NUI Galway

1. Graphs and Graph Theory

Lecture 5: Paths, Trees and Algorithms

Sequences of interconnected edges in a graph are called **paths**, leading to notions of **connectivity** and **distance**. A **tree** is a particularly useful kind of connected graph, that is frequently used as a data structure in Computer Science.

We will study **random trees** and some classical algorithms for tree traversal. A network has the structure of a tree when it is of a hierarchical nature, like an ancestry chart (https://en.wikipedia.org/wiki/Pedigree_chart) or a **river network**.

[Image from EPA Maps (https://gis.epa.ie/EPAMaps/)]

In [1]:
```
import networkx as nx
import numpy as np
```

Paths

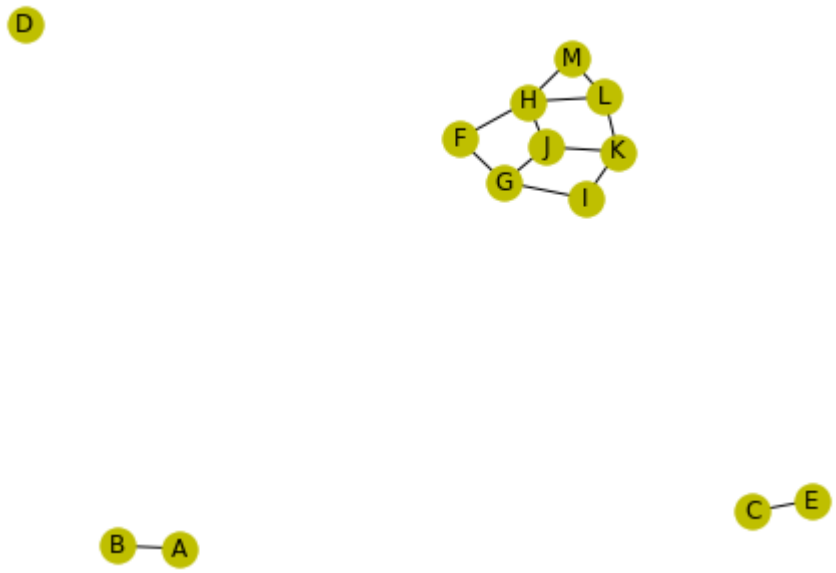
The fundamental notion of **connectivity** in a network is closely related to the notion of **paths** in a graph.

Definitions.

- A **path** in a graph $G = (X, E)$ is a sequence of nodes, where any pair of consecutive nodes in the sequence is (linked by) an edge in E.
- Such a path can have repeated nodes. If it doesn't, the path is called a **simple path**.
- The **length** of a path is the number of edges it involves (that is the number of nodes minus 1).
- At each vertex $x \in X$, there is a unique path of length 0, the **empty path**, consisting of vertex x only.
- A **cycle** is a path of length at least 3 that is a simple path, except for the first and the last node being the same.

In [2]:
```
nodes = 'ABCDEFGHIJKLM'
edges = [
    'AB', 'CE', 'FG', 'FH', 'GI', 'GJ', 'HJ', 'HL', 'HM',
    'IK', 'JK', 'KL', 'LM'
]
G = nx.Graph()
G.add_nodes_from(nodes)
G.add_edges_from(edges)
```

In [3]:
```
opts = { "with_labels": True, "node_color": 'y'}
nx.draw(G, **opts)
```



- (F, G, I) is a path in the graph above, and (H, J, K, L, H) is a cycle.

- A cycle in a simple graph provides, for any two nodes on that cycle, (at least) two different paths from one to the other.
- Note that each edge (and node) of the 1970 Internet graph belongs to a cycle. This makes the other way around the cycle an alternative route in case one of the edges should fail.

- (In a *directed* network, paths are directed, too. A path from a vertex x to a vertex y is a sequence of vertices $x = x_0, x_1, \dots, x_k = y$ such that, for any $i = 1, \dots, k$, there is an edge from x_{i-1} to x_i in the graph.)

Connected Components

Communication and transportation networks tend to be connected, as this is their main purpose: to connect.

Definition.

- A simple graph is **connected** if, for every pair of nodes, there is a path between them.
- If a graph is not connected, it naturally breaks into pieces, its **connected components**.

- The connected components of the graph above are the node sets $\{A, B\}$, $\{C, E\}$, $\{D\}$, and $\{F, G, H, I, J, K, L, M\}$.
- Note that a component can consist of a single node only.

In [4]:
```
list(nx.connected_components(G))
```

Out[4]:
```
[{'A', 'B'}, {'C', 'E'}, {'D'}, {'F', 'G', 'H', 'I', 'J', 'K', 'L',
'M'}]
```

Note. The relation 'there is a **path** from x to y on the node set X of a graph is the **transitive closure** of the graph relation 'there is an **edge** between x and y'. It is

- **reflexive** (as each node x is connected to itself by the zero length path starting and ending at x),
- **symmetric** (as a path from x to y can be used backwards as a path from y to x),
- and **transitive** (as a path from x to y and a path from y to z together make up a path from x to z),

hence an **equivalence relation**. The connected components of the graph are the parts (equivalence classes) of the corresponding **partition** of X.

Trees

- A graph is called **acyclic** if it does not contain any cycles.

- A **tree** is a (simple) graph that is **connected** and **acyclic**.

In other words, between any two vertices in a tree there is **exactly one simple path**.

Trees can be characterized in many different ways.

Theorem. Let $G = (X, E)$ be a (simple) graph of order $n = |X|$ and size $m = |E|$. Then the following are equivalent:

- G is a tree (i.e. acyclic and connected);
- G is connected and $m = n - 1$;
- G is a minimally connected graph (i.e., removing any edge will disconnect G);
- G is acyclic and $m = n - 1$;
- G is a maximally acyclic graph (i.e., adding any edge will introduce a cycle in G).

Random Trees

Theorem (Cayley's Formula). There are exactly n^{n-2} distinct (labelled) trees on the n-element vertex set $X = \{0, 1, 2, \dots, n-1\}$, if $n > 1$.

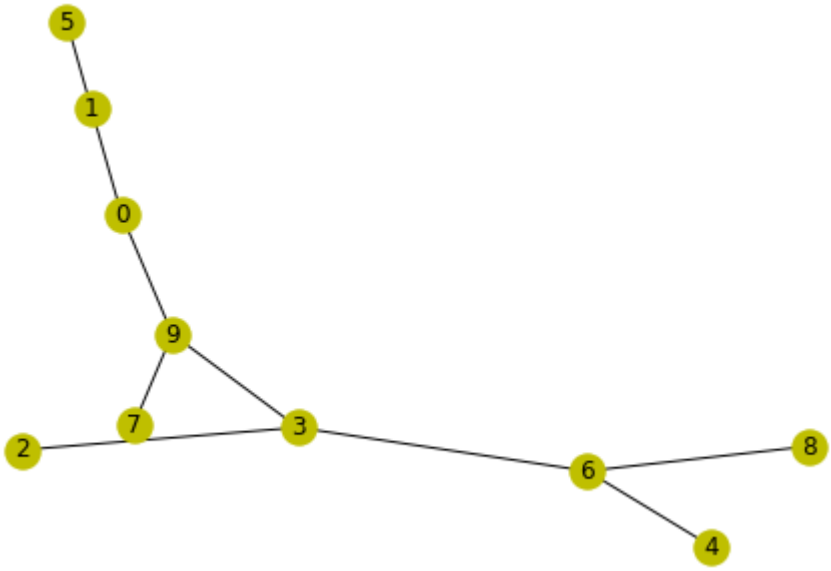
Also, there is one (trivial) tree on 1 vertex, but there is no tree on 0 vertices. Some values for $n > 1$:

In [5]:
```
domain = range(2, 10)
print(np.array([domain, [n**(n-2) for n in domain]]))
```

```
[[      2       3       4       5       6       7       8       9]
 [      1       3      16     125    1296   16807  262144 4782969]]
```

Proof. Each tree on $X = \{0, 1, 2, \dots, n-1\}$ corresponds to a unique sequence of $n - 2$ elements of X, its **Prüfer Code** (https://en.wikipedia.org/wiki/Pr%C3%BCfer_sequence).

In [6]:
```
n = 10
TT = nx.random_tree(n)
nx.draw(TT, **opts)
```



How to determine the Prüfer code of a tree T (destructively):

- Find the smallest leaf x
- Record the label y of its unique neighbor
- Remove x (and the edge $x - y$) from T
- Repeat until T has only 2 nodes left.

In [7]:
```
def pruefer_list(tree):
    for x in tree:
        if tree.degree(x) == 1:
            for y in tree[x]:
                tree.remove_node(x)
                return [x, y]
```

In [8]:
```
T = TT.copy()
code = [pruefer_list(T) for k in range(n-2)]
```

In [9]:
```
print(np.array(code).transpose())
```

```
[[2 4 5 1 0 7 8 6]
 [3 6 1 0 9 9 6 3]]
```

This process destroys the tree T almost completely.

In [10]:
```
print(T.nodes())
print(T.edges())
```

```
[3, 9]
[(3, 9)]
```

Let's wrap this up as a python function

In [11]:
```
def pruefer_node(tree):
    for x in tree:
        if tree.degree(x) == 1:
            for y in tree[x]:
                tree.remove_node(x)
                return y
```

In [12]:
```
def pruefer_code(tree):
    return [pruefer_node(tree) for k in range(tree.order() - 2)]
```

In [13]:
```
T = TT.copy()
code = pruefer_code(T)
code
```

Out[13]: [3, 6, 1, 0, 9, 9, 6, 3]

Maybe surprisingly, the tree can be reconstructed from its Prüfer code. This is based on the following fact and shows that the map from trees to codes is a bijection!

> **Fact:** The degree of node x is 1 plus the number of entries x in the Prüfer code of T.

In [14]:
```
degrees = [1 for k in range(n)]
for k in code:
    degrees[k] += 1
degrees
```

Out[14]: [2, 2, 1, 3, 1, 1, 3, 1, 1, 3]

In [15]:
```
[TT.degree[x] for x in TT]
```

Out[15]:
```
[2, 2, 1, 3, 1, 1, 3, 1, 1, 3]
```

How to restore the tree from its Prüfer code:

- Start with a graph with vertex set $X = \{0, 1, 2, \ldots, n-1\}$ (and no edges yet).
- Compute the desired node degrees from the code.
- For each node y in the code find the smallest degree 1 node x and add the edge $x - y$, then decrease the degrees of both x and y by 1.
- Finally, connect the remaining 2 nodes of degrees 1 by an edge.

In [16]:
```
T = nx.empty_graph(n)
nx.draw(T, **opts)
```

In [17]:
```
code
```

Out[17]:
```
[3, 6, 1, 0, 9, 9, 6, 3]
```

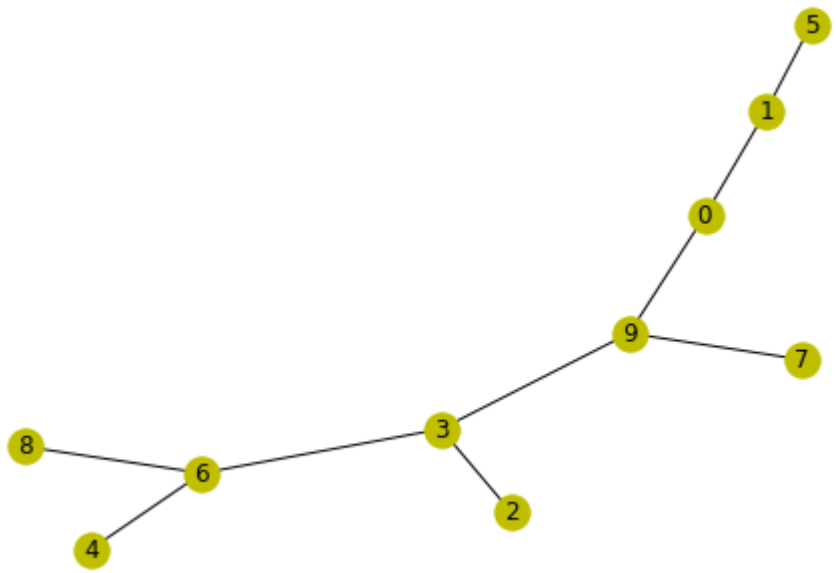
In [18]:
```
# repeat n-2 times:
for y in code:
    x = degrees.index(1)
    T.add_edge(x, y)
    degrees[x] -= 1;  degrees[y] -= 1
    print(degrees, ": edge", x, "--", y)
```

```
[2, 2, 0, 2, 1, 1, 3, 1, 1, 3] : edge 2 -- 3
[2, 2, 0, 2, 0, 1, 2, 1, 1, 3] : edge 4 -- 6
[2, 1, 0, 2, 0, 0, 2, 1, 1, 3] : edge 5 -- 1
[1, 0, 0, 2, 0, 0, 2, 1, 1, 3] : edge 1 -- 0
[0, 0, 0, 2, 0, 0, 2, 1, 1, 2] : edge 0 -- 9
[0, 0, 0, 2, 0, 0, 2, 0, 1, 1] : edge 7 -- 9
[0, 0, 0, 2, 0, 0, 1, 0, 0, 1] : edge 8 -- 6
[0, 0, 0, 1, 0, 0, 0, 0, 0, 1] : edge 6 -- 3
```

Add the final edge:

In [19]:
```
e = [x for x in range(n) if degrees[x] == 1]
T.add_edge(*e)
print(e)
nx.draw(T, **opts)
```

```
[3, 9]
```



Turn the entire procedure into a `python` function:

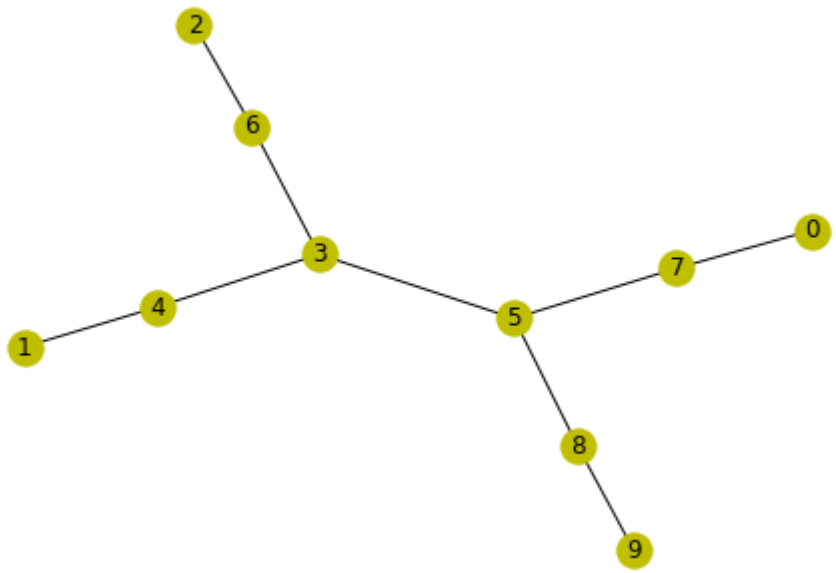
In [20]:
```
def tree_pruefer(code):

    # initialize graph and defects
    n = len(code) + 2
    tree = nx.empty_graph(n)
    degrees = [1 for x in tree]
    for y in code:
        degrees[y] += 1

    # add edges
    for y in code:
        for x in tree:
            if degrees[x] == 1:
                tree.add_edge(x, y)
                for z in (x, y):
                    degrees[z] -= 1
                break

    # final edge
    e = [x for x in tree if degrees[x] == 1]
    tree.add_edge(*e)

    return tree
```

- We can now construct a random tree on n nodes from a random Prüfer code of length $n-2$.

In [21]:
```
code = np.random.randint(n, size=n-2)
code
```

Out[21]:
```
array([7, 4, 6, 3, 3, 5, 5, 8])
```

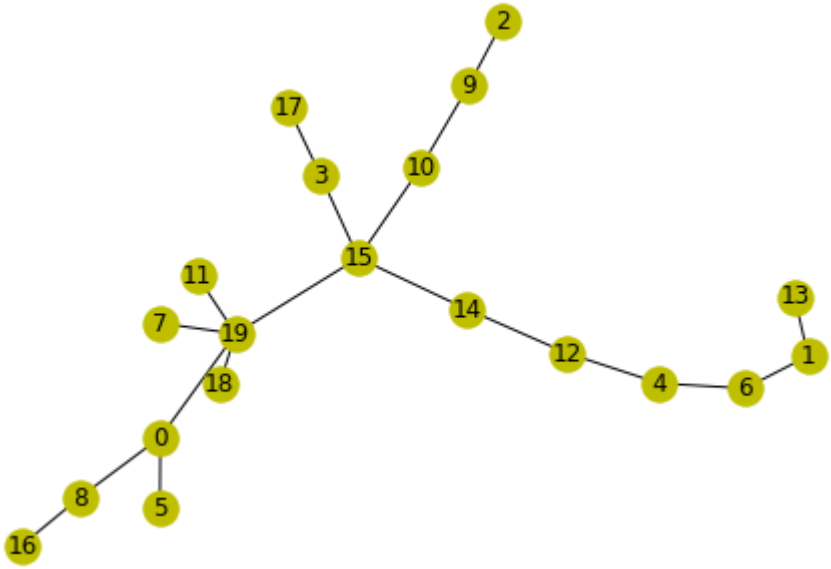
In [22]:
```
tree = tree_pruefer(code)
nx.draw(tree, **opts)
```



Finally, we wrap this up into a `python` function `random_tree` .

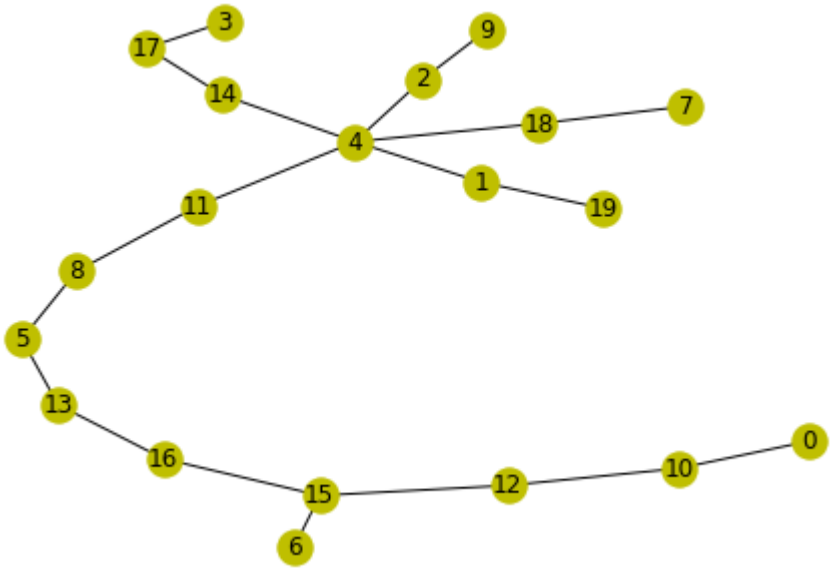
In [23]:
```
def random_tree(n):
    code = np.random.randint(n, size=n-2)
    return tree_pruefer(code)
```

In [24]:
```
T = random_tree(20)
nx.draw(T, **opts)
```



Of course, `networkx` has its own random trees:

In [25]:
```
T = nx.random_tree(20)
nx.draw(T, **opts)
```



Depth First Search

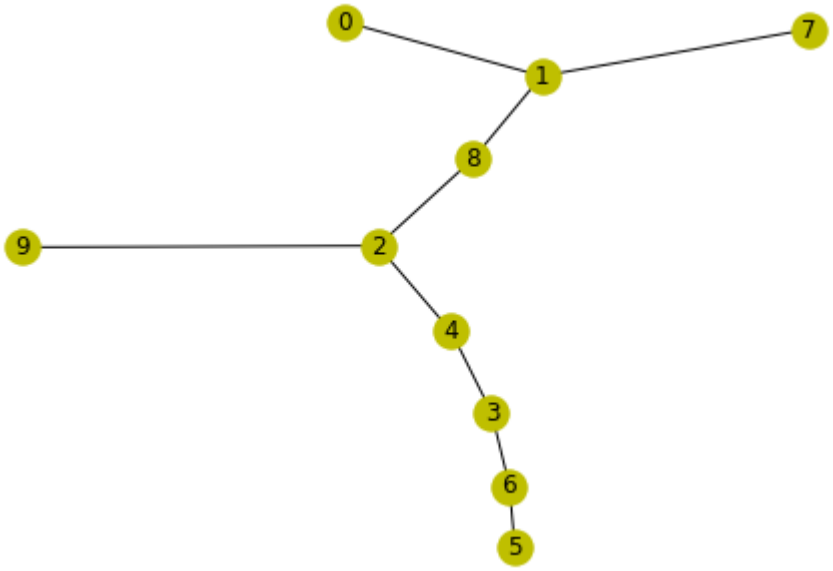
DFS (https://en.wikipedia.org/wiki/Depth-first_search) and BFS (https://en.wikipedia.org/wiki/Breadth-first_search) are simple but efficient tree (and graph) traversal algorithms.

DFS: Given a rooted tree T with root x, visit all nodes in the tree.

- $S \leftarrow (x)$
- while $S \neq \emptyset$:
- $y \leftarrow S$.pop()
- visit(y)
- S.push(y.children)

Here S is a **stack** (LIFO): S.pop() yields the **newest** entry.

In [26]:
```
TT = random_tree(n)
nx.draw(TT, **opts)
```



In [27]:
```
T = TT.copy()
x = 0
stack = [x]
while len(stack) > 0:
    y = stack.pop()
    stack.extend(T[y])
    T.remove_node(y)
    print(y, stack)
```

```
0 [1]
1 [7, 8]
8 [7, 2]
2 [7, 4, 9]
9 [7, 4]
4 [7, 3]
3 [7, 6]
6 [7, 5]
5 [7]
7 []
```

Breadth First Search

BFS: Given a rooted tree T with root x, visit all nodes in the tree.

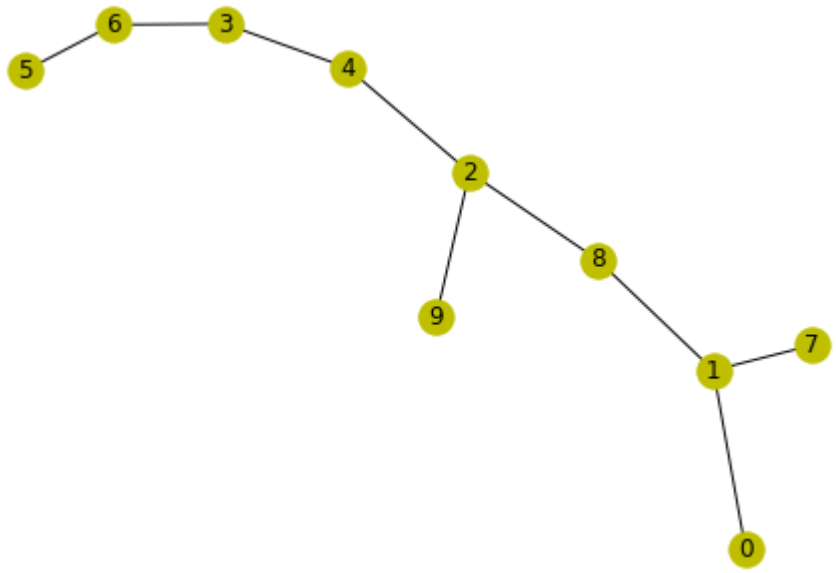
- $Q \leftarrow (x)$
- while $Q \neq \emptyset$:
- $y \leftarrow Q$.pop()
- visit(y)
- Q.push(y.children)

Here, Q is a **queue** (https://en.wikipedia.org/wiki/Queue_(abstract_data_type)) (FIFO): Q.pop() yields the **oldest** entry.

In [28]:
```
T = TT.copy()
x = 0
queue = [x]
while len(queue) > 0:
    y = queue.pop(0)
    queue.extend(T[y])
    T.remove_node(y)
    print(y, queue)
```

```
0 [1]
1 [7, 8]
7 [8]
8 [2]
2 [4, 9]
4 [9, 3]
9 [3]
3 [6]
6 [5]
5 []
```

In [29]:
```
nx.draw(TT, **opts)
```



Code Corner

python

- `[].pop` [doc] (https://docs.python.org/2/tutorial/datastructures.html)
- `[].extend` [doc] (https://docs.python.org/2/tutorial/datastructures.html)

networkx

- `connected_components` [doc] (https://networkx.github.io/documentation/stable/reference/algorithms/component.html)
- `random_tree` [doc] (https://networkx.github.io/documentation/stable/reference/generated/networkx.generators.tre
- `copy` : [doc] (https://networkx.org/documentation/stable/reference/classes/generated/networkx.Graph.copy
- `empty_graph` [doc] (https://networkx.github.io/documentation/stable/reference/generated/networkx.generators.cla

numpy

- `array` : [doc] (https://numpy.org/doc/stable/reference/generated/numpy.array.html)
- `transpose` : [doc] (https://numpy.org/doc/stable/reference/generated/numpy.transpose.html)

- `random.randint` : [doc] (https://numpy.org/doc/stable/reference/random/generated/numpy.random.randint.html)

Exercises

1. A tree T uniquely determines its Prüfer code, and hence the two nodes that remain after (destructively) computing the code. What are those two nodes, in terms of properties of T, or its Prüfer code?
2. What tree has Prüfer code $(0, 1, 2, \ldots, n-3)$?

In [ ]: